

Paradigmi di Programmazione - A.A. 2022-23

Esame Scritto del 13/01/2022

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$(\lambda x. \lambda y. (xy))(\lambda x. \lambda z. (yx))y$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

SOLUZIONE:

$$\begin{aligned} & (\lambda x. \lambda y. (xy))(\lambda x. \lambda z. (yx))y \\ \equiv_{\alpha} & \underline{(\lambda x. \lambda k. (xk))(\lambda x. \lambda z. (yx))y} \\ \rightarrow & \underline{(\lambda k. ((\lambda x. \lambda z. (yx))k))y} \\ \rightarrow & \underline{(\lambda k. ((\lambda z. (yk))))y} \\ \rightarrow & \underline{(\lambda z. (yy))} \end{aligned}$$

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml mostrando i passi fatti per inferirlo.

```
fun f -> fun x -> match (f x) with
  | (0,g) -> g x
  | (n,g) -> g n;;
```

SOLUZIONE:

Struttura del tipo:

`F -> X -> RIS`

Uso per convenzione `F`, `X` e `G` come variabili di tipo per le variabili `f`, `x` e `g`, e `RIS` come

Vincoli:

```
F = X -> A      (da pattern matching)
A = int * G      (da pattern nel primo caso del p.m.)
G = X -> RIS     (da primo caso del p.m.)
      G = int -> RIS (da secondo caso del p.m.)
```

Ne consegue:

```
X = int
F = int -> (int * (int -> RIS))
RIS = RIS
```

Tipo inferito:

```
(int -> (int * (int -> RIS))) -> int -> RIS
```

che in sintassi OCaml corrisponde a:

```
(int -> (int * (int -> 'a))) -> int -> 'a
```

Esercizio 3 [Punti 7]

Si considerino i seguenti tipi di dato che consentono di rappresentare rispettivamente punti e rettangoli in un piano cartesiano tramite dei record:

```
type punto = {x:float; y:float};;
type rettangolo = { id:int; p1:punto; p2:punto};;
```

In particolare, un rettangolo è rappresentato tramite un identificativo intero, e due punti che rappresentano l'angolo in basso a destra (`p1`) e in alto a sinistra (`p2`) del rettangolo rappresentato, ad esempio:

```
let r1 = {id=123; p1={x=1.;y=1.}; p2={x=6.;y=2.}};;
let r2 = {id=44; p1={x=1.;y=2.}; p2={x=2.;y=3.}};;
let r3 = {id=332; p1={x=4.;y=2.}; p2={x=6.;y=4.}};;
```

Tramite questa rappresentazione, è possibile definire funzioni per calcolare la base, l'altezza e l'area di un rettangolo come segue:

```
let base r = r.p2.x -. r.p1.x;;
let altezza r = r.p2.y -. r.p1.y;;
let area r = base r *. altezza r;;
```

Ad esempio:

```
area r1;;      (* restituisce 5. *)
area r2;;      (* restituisce 1. *)
area r3;;      (* restituisce 4. *)
```

Una figura complessa formata da più rettangoli può essere rappresentata come una lista di questi record, ossia come un valore del tipo `rettangolo list`.

Si definiscano, usando i costrutti di programmazione funzionale di OCaml, le funzioni `check_crescenti` e `swap` con i seguenti tipi

```
check_crescenti : rettangolo list -> bool
```

```
swap : int -> int -> rettangolo list -> rettangolo list
```

tali che:

- data una lista di rettangoli, `check_crescenti` restituisce `true` se i rettangoli della lista sono ordinati per area crescente. Ad esempio, `check_crescenti [r1; r2; r3]` restituisce `false`, mentre `check_crescenti [r2; r3; r1]` restituisce `true`;
- dati due interi `id1` e `id2` e una lista di rettangoli `lis`, la funzione `swap id1 id2 lis` restituisce una lista ottenuta scambiando di posizione i rettangoli con identificatori `id1` e `id2` in `lis`. Ad esempio, `swap 123 332 [r1;r2;r3]` restituisce `[r3;r2;r1]`.

Si può assumere che gli identificatori dei rettangoli nella lista siano tutti diversi (senza controllarlo). Inoltre, se la lista non contiene rettangoli con identificatori `id1` e `id2` deve essere sollevata un'eccezione.

SOLUZIONE:

Una possibile soluzione:

```
let check_crescenti lis =
  let rec check lis last_area =
    match lis with
    | [] -> true
    | r::lis' -> let a = area r in
                  if a >= last_area then check lis' a
                  else false
  in check lis 0.;;
```

(* oppure *)

```
let check_crescenti lis =
  let f (last_area, ok) r =
    let a = area r in
    if a >= last_area then (a,ok) else (a,false)
  in
  let (a,ok) = List.fold_left f (0.,true) lis
  in ok;;
```

```
let swap id1 id2 lis =
  let rec get id lis =
    match lis with
    | [] -> failwith "non trovato"
    | r::lis' -> if r.id=id then r else get id lis'
  in
  let rec swap_map r1 r2 =
    let f r = if r.id=r1.id then r2 else
              if r.id=r2.id then r1 else
              r
    in List.map f lis
  in swap_map (get id1 lis) (get id2 lis);;
```

(* oppure *)

```
let swap id1 id2 lis =
  let rec get id lis =
    match lis with
    | [] -> failwith "non trovato"
    | r::lis' -> if r.id=id then r else get id lis'
  in
  let rec swap_rec r1 r2 lis =
    match lis with
    | [] -> []
    | r::lis' -> if r.id=r1.id then r2::(swap_rec r1 r2 lis') else
                  if r.id=r2.id then r1::(swap_rec r1 r2 lis') else
                  r::(swap_rec r1 r2 lis')
  in
  let r1 = get id1 lis in
  let r2 = get id2 lis in
  swap_rec r1 r2 lis;;
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml con un costrutto per la definizione di **funzioni ricorsive limitate**, ossia funzioni che si possono chiamare ricorsivamente per un numero limitato di volte. Il limite deve essere specificato al momento della definizione della funzione. Quando si va ad applicare la funzione, se, durante la sua esecuzione, il numero delle chiamate ricorsive che vengono effettuate supera il limite, l'interprete termina sollevando un'eccezione. Se invece il numero delle chiamate ricorsive non raggiunge il limite, l'interprete completa correttamente il calcolo.

Si consideri il seguente esempio in una sintassi concreta ispirata da quella di OCaml, dove il nuovo costrutto si chiama `let reclim`, mentre `fact` è il nome della funzione, `n` è il parametro (di cui calcolare il fattoriale) e 5 è il limite alle chiamate ricorsive che possono essere fatte per calcolare il risultato:

```
let reclim fact n 5 = if n=0 then 1 else n*fact (n-1)
in fact 3
```

il risultato in questo caso è 6. Analogamente

```
let reclim fact n 5 = if n=0 then 1 else n*fact (n-1)
in fact 4
```

termina con il risultato corretto, ossia 24. Invece

```
let reclim fact n 5 = if n=0 then 1 else n*fact (n-1)
in fact 5
```

porta ad una eccezione, in quanto per calcolare il fattoriale di 5 servono 6 chiamate ricorsive, che sono più del limite specificato nella definizione della funzione.

Attenzione, il limite specificato nel momento della definizione deve essere un valore maggiore di 0. In caso contrario l'interprete si interrompe immediatamente sollevando un'eccezione.

SOLUZIONE:

Una possibile soluzione:

```
type exp = ...
  | LetrecLim of ide * ide * exp * exp * exp

type evT = ...
  | RecClosureLim of ide * ide * int * exp * evT env

let rec eval e s = match e with
  ...
  | LetrecLim (f, arg, cont, body, e) ->
    let num = (match eval cont s with
      | Int(n) -> if n>0 then n else failwith "Limite non positivo"
      | _ -> failwith "Limite non numerico"
    )
    in let newenv = bind s f (RecClosureLim(f, arg, num, body, s))
    in eval e newenv
  | Apply (e1,e2) ->
    let fclosure = eval e1 s in
    (match fclosure with
    ...
    | RecClosureLim(f, arg, num, body, fDecEnv) ->
      if num>0 then
        let aVal = eval e2 s in
        let s1 = bind s f (RecClosureLim(f, arg, (num-1), body, fDecEnv) in
        let s2 = bind s1 arg aVal in
        eval body s2
      else
        failwith "Ecceduto limite ricorsione"
    )
  )
```